

Mobile-GS: Real-Time Gaussian Splatting For Mobile Devices

2026 ICLR

“Xiaobiao Du^{1, 3}, Yida Wang³, KunZhan³, XinYu^{2*}”

¹University of Technology Sydney, ²Adelaide University, ³Li Auto Inc

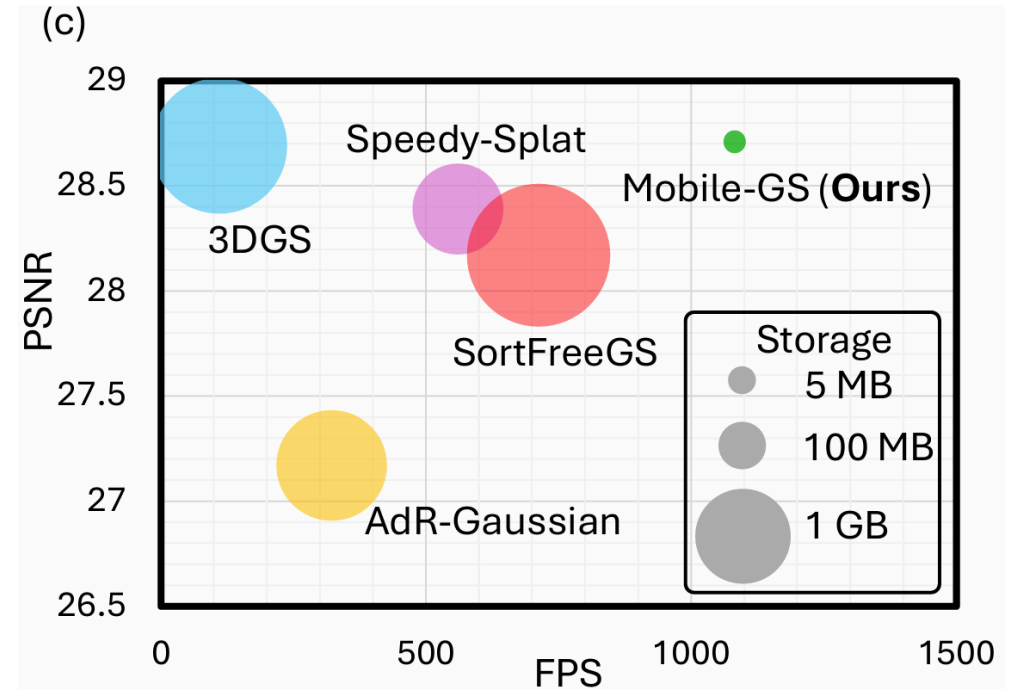
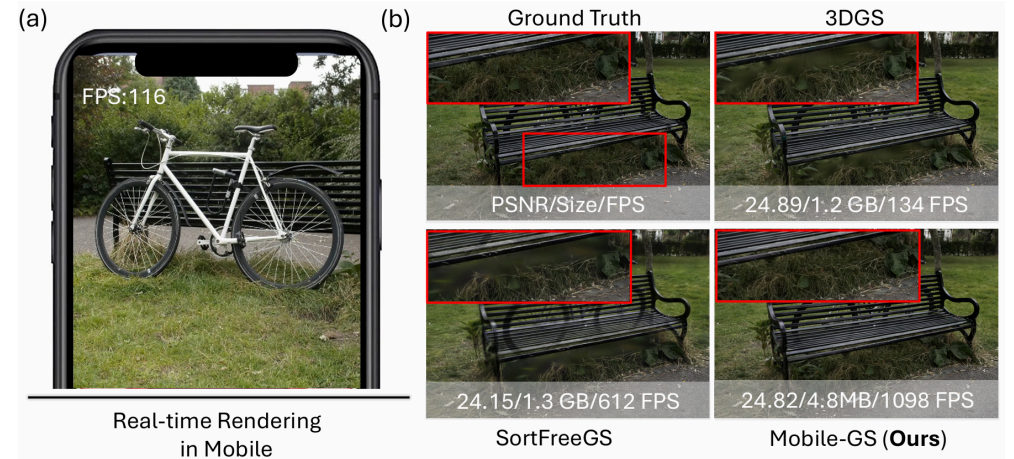
2026.03.19

Contents

1. Introduction
2. Related Work
3. Method
4. Experiments
5. Conclusion

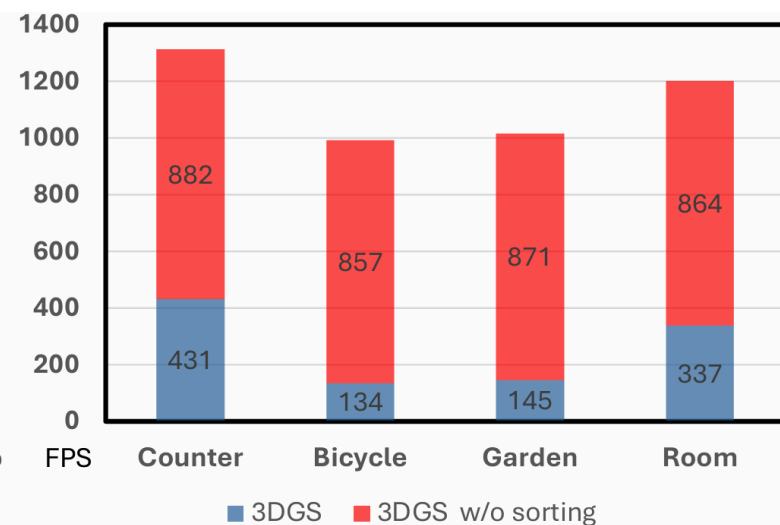
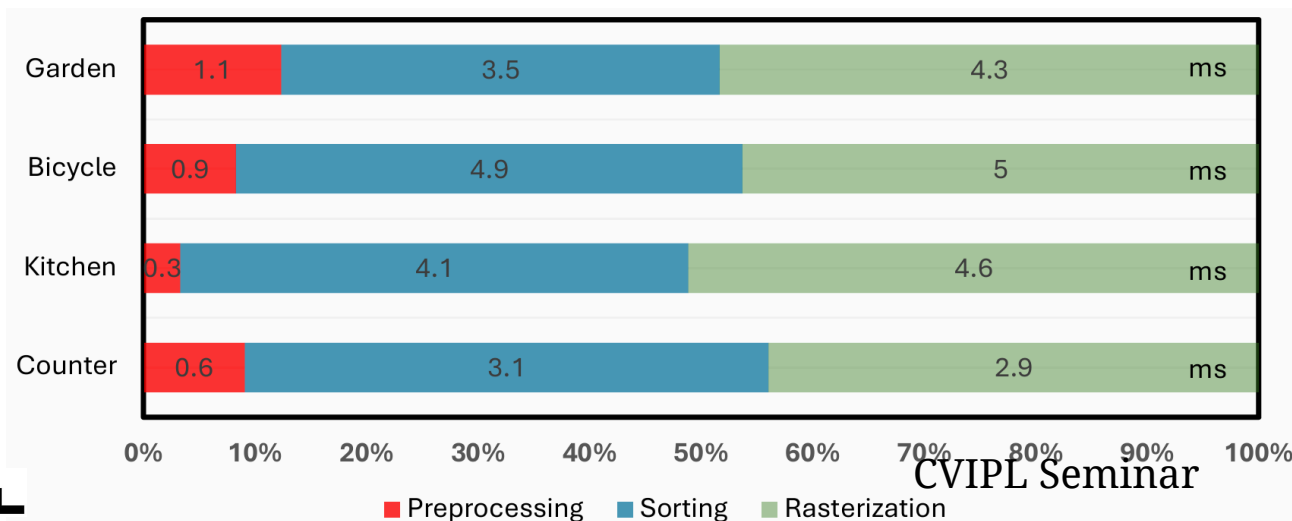
Introduction

- Gaussian Splatting은 빠르지만, 전체 pipeline에서의 병목 존재
- 3DGS의 지속적인 병목: **Sorting**
 - 3DGS는 alpha blending을 통해 픽셀의 색상을 결정하는데, 이 때, near-to-far depth sorting이 필요함
 - 따라서, 3DGS의 병목은 Sorting이며, 이는 모바일에서 실시간 렌더링을 어렵게 하는 주요 원인임
 - 즉, sorting이 비싼연산임



Introduction

- 본 논문은 병목을 줄이고 추가 최적화를 통해 모바일 실시간 렌더링을 달성하는 것이 목표
- Mobile-GS는 다음과 같은 세 가지 핵심 아이디어로 구성
 - Order-free rendering : Sorting 제거 (Depth-aware order-independent rendering)
 - Quantization : 메모리 절약을 (Neural vector quantization + SH distillation)
 - Fewer Gaussian points : 퀄리티 유지하면서 Gaussian point 수 줄이기 (Contribution-based pruning)



Related Work

3D Gaussian Splatting

- 3DGS (Kerbl et al., 2023) : 장면을 anisotropic 3D Gaussian으로 표현하고, tile-based differentiable rasterizer를 사용해 novel view를 렌더링함.
- Scaffold-GS (Lu et al., 2024) : hierarchical scaffold structure를 도입하여, 시각적 품질을 유지하면서 Gaussian 수를 줄이는 방식으로 고품질 렌더링을 달성함.
- Mini-Splatting (Fang & Wang, 2024) : pruning + densification 전략을 통해, 매우 compact한 Gaussian 구조를 생성하는 데 초점을 맞춤.
- 후속 compression 계열 연구들 (Chen et al., 2024b; 2025; Wang et al., 2024b; Liu et al., 2024) : 더 lightweight한 representation을 만들기 위해 compression ratio를 높이는 방향으로 접근함.
- MVGS (Du et al., 2026) : 최적화 단계에서 multi-view learning을 도입하여, 3DGS의 multi-view constraint를 강화하고 전체 렌더링 성능을 향상시킴.

Order Independent Transparency (OIT)/Sort-Free 3DGS

- Traditional OIT methods
 - A-buffer, K-buffer, Depth Peeling, Stochastic Transparency → sorting 없이 transparency를 처리하기 위한 전통적 접근
- Recent sort-free 3DGS works
 - approximated compositing 또는 order-independent accumulation을 통해 sorting 제거
- Representative work: SortFreeGS(Hou et al., 2025)
 - Weighted sum 기반 compositing으로 order-independent rendering 수행

$$C = \frac{c_{bg}w_{bg} + \sum_{i=1}^N c_i\alpha_iw(d_i)}{w_{bg} + \sum_{i=1}^N \alpha_iw(d_i)}, \quad w(d_i) = \exp(-\sigma_i d_i^{\beta_i})$$

- 그럼에도 불구하고 용량이 너무크고, inference time 측면에서 여전히 delay가 존재
 - 이와 관련해서 압축과 경량화에 집중한 후속 연구들이 수행됨

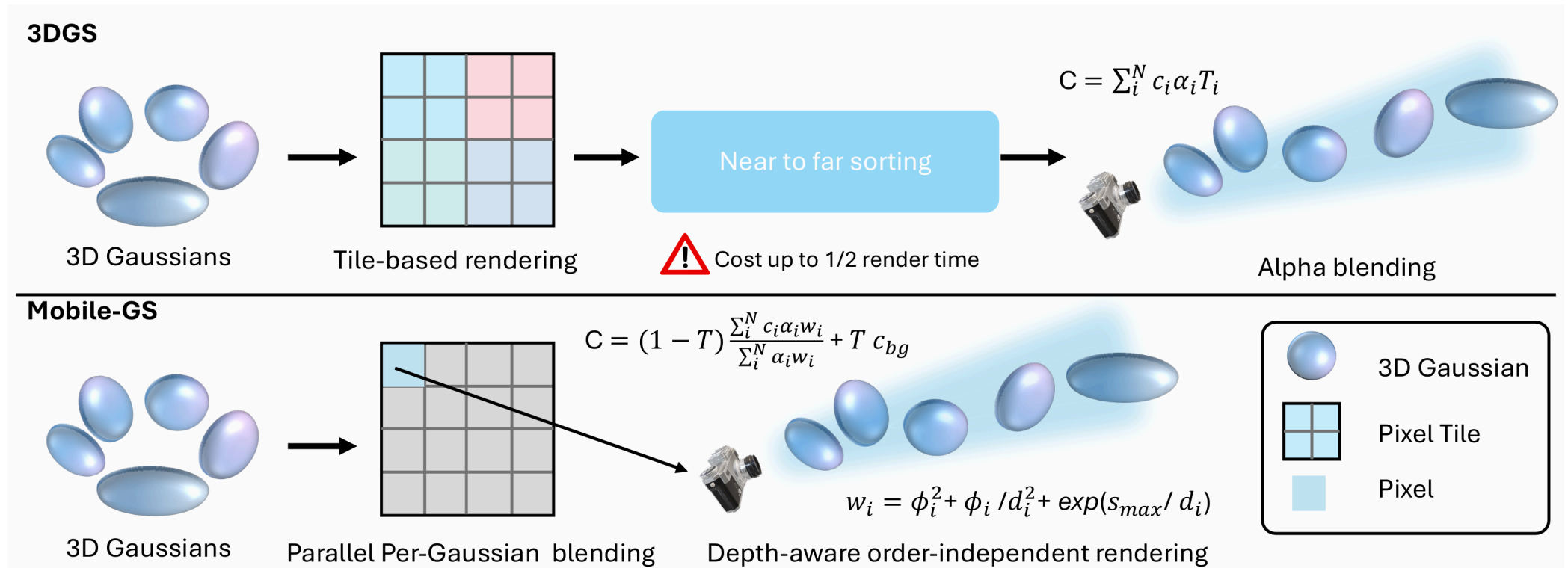
Related Work

Gaussian Compression and Pruning.

- Compression methods
 - vector quantization / entropy encoding을 통해 Gaussian representation 압축
 - locality-aware strategy로 Gaussian attributes를 compact local representation으로 압축
- LODGE / MaskGaussian / GaussianSpa
 - importance, contribution, sparsity constraint 기반 pruning으로 Gaussian 수 감소
- Redundancy(중복) 감소에는 효과적이지만, 렌더링 품질 저하와 여전한 storage overhead 문제가 존재
- 따라서, Mobile-GS는 **3가지 contribution**(order free, quantization, pruning) 모두를 활용하여, **모바일에서 실시간 렌더링**이 가능한 compact한 Gaussian Splatting을 목표로 함

Method

Overview of Mobile-GS



Method

I. Depth-aware Order-Independent Rendering

- 앞서 언급한 alpha-blending, 기존의 sort-free 3DGS 방식들과 다르게, 본 논문은 아래의 수식으로 order-independent rendering을 수행
 - Alpha blending이긴 함
 - 다만, 여기서는 foreground와 background의 비율을 조정할 수 있는 Global transmittance인 T 를 도입하여 사용

$$C = (1 - T) \frac{\sum_{i=1}^N c_i \alpha_i w_i}{\sum_{i=1}^N \alpha_i w_i} + T c_{bg}, \quad T = \prod_{j=1}^N (1 - \alpha_j)$$

- 여기서 opacity와 weight로 normalize해줌으로써 결국 color는 weighted average로 계산

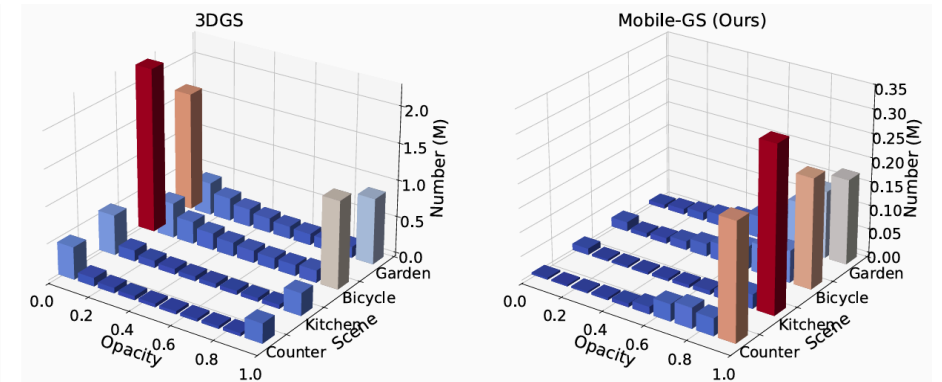
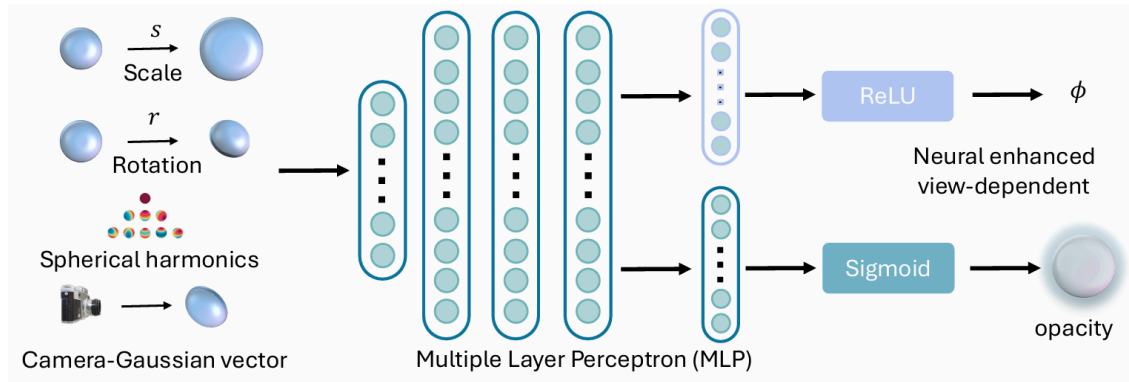
Method

- Weighted average, 즉 덧셈으로 pixel의 색상을 계산하기 위해서, alpha와 weight를 다음과 같이 정의

$$\alpha_i = o_i \exp\left(-\frac{1}{2} \Delta x_i^\top \Sigma_i^{-1} \Delta x_i\right), \quad w_i = \phi_i^2 + \frac{\phi_i}{d_i^2} + \exp\left(\frac{s_{\max}}{d_i}\right)$$

- feature를 통해 ϕ_i 와 o_i 를 MLP로 예측(Neural View-dependent Enhancement):

$$F = \text{MLP}_f(\mathbf{p}_i, s_i, r_i, Y_i), \quad \phi_i = \text{ReLU}(\text{MLP}_\phi(F)), \quad o_i = \sigma(\text{MLP}_o(F)), \quad \mathbf{p}_i = \frac{\mu_i - \mathbf{t}_v}{\|\mu_i - \mathbf{t}_v\|}$$



Method

II. Distillation and Quantization

- First-order Spherical Harmonics Distillation

- SH coefficient 마저도 압축하여 경량화에 신경을 더 쓰겠다
- 기존 3DGS에서는 3차까지 쓰고, LightGS에서는 2차만 썼는데, 1차까지 3개 총 4개로 줄여 표현
- Teacher-student distillation
- 여기서 teacher는 Mini-Splatting 모델로, student는 Mobile-GS 모델

$$\mathcal{L}_{\text{distill}} = \frac{1}{|P|} \sum_{p \in P} \|C_p^{\text{tea}} - C_p\|$$

- Geometry 안정화를 위해 depth distillation loss도 추가

$$\mathcal{L}_{\text{depth}}(\mathbf{D}, \mathbf{D}^{\text{tea}}) = \frac{1}{|P|} \sum_{p \in P} \left(\log(\hat{D}_p) - \log(\hat{D}_p^{\text{tea}}) \right)^2 - \frac{1}{|P|^2} \left(\sum_{p \in P} \left(\log(\hat{D}_p) - \log(\hat{D}_p^{\text{tea}}) \right) \right)^2$$

Method

- Neural Vector Quantization

- 기존의 방식들보다도 많은 경량화를 이뤘지만, mobile device에서 랜더링하기 위해서는 좀 더..
- Gaussian attributes를 Neural vector quantization을 통해 압축

- Gaussian 하나의 attributes들을 K개의 sub-vector로 분할

$$z = [z_1, z_2, \dots, z_K], \quad z \in \mathbb{R}^{K \times L}$$

- 각 sub-vector는 codebook에서 가장 가까운 벡터로 양자화

$$C^K \in \mathbb{R}^{B \times L}$$

- ...?

Method

- For example...

- Gaussian 하나의 attribute를 12차원이라고 해보자.

$$z \in \mathbb{R}^{12}$$

- 이걸 subvector로 나누는데, $K = 3$ subvectors로 나눈다고 해보자.

$$z = [z_1, z_2, z_3], \quad z_k \in \mathbb{R}^4$$

- 수십, 수백만개의 Gaussian들이 있을텐데, K-means clustering을 통해서, 각 subvector에 대해서 codebook을 생성 ($k = B$)

$$C^k \in \mathbb{R}^{B \times L}$$

- 그런데, z 내부에는 다차원의 SH계수를 0차원 색상인 $h_d \in \mathbb{R}^1$ 와 view dependent color인 $h_v \in \mathbb{R}^3$ 로 압축한걸 들고있다
- 랜더링할 때에는 이를 MLP로 다시 예측해서 사용

$$f_d = \text{MLP}_d(h_d, h_v), \quad f_v = \text{MLP}_v(h_d, h_v)$$

- 이때 복원된 색상정보가 앞서 보았던 MLP에 다시 들어가 ϕ_i 와 o_i 를 예측하는데 사용됨

Method

III. Contribution-based Pruning

- 너무 투명하거나, scale이 너무 작은 gaussian은 랜더링에 거의 기여하지 않음
 - 다른말로 해석해보면, 오히려 scene 정확도를 해치는 floating point 관리
 - Q 함수를 사용하는 이유는, 항상 하위 몇%를 유지할 수 있도록 함

$$\mathcal{C}_{\text{opacity}}^{(t)} = \{g \in \mathcal{G} \mid o_g < Q_\tau(o)\}, \quad \mathcal{C}_{\text{scale}}^{(t)} = \{g \in \mathcal{G} \mid s_{\max}(g) < Q_\tau(s_{\max})\}$$

$$\mathcal{C}_{\text{prune}}^{(t)} = \mathcal{C}_{\text{opacity}}^{(t)} \cap \mathcal{C}_{\text{scale}}^{(t)}$$

- 다만, $\mathcal{C}_{\text{prune}}^{(t)}$ 에 해당하는 녀석들을 바로 pruning하지 않고, 투표를 함
 - $I_{\text{prune}} = 100$ 은 pruning interval, $v = 60$ 는 pruning vote threshold, 즉 100번 중 60번 이상 prune해야되면 지움

$$V_g^{(t+1)} = V_g^{(t)} + \mathbf{1}[g \in \mathcal{C}_{\text{prune}}^{(t)}], \quad \mathcal{G}_{\text{new}} = \mathcal{G} \setminus \{g \in \mathcal{G} \mid \mathbf{1}[V_g^{(t)} > I_{\text{prune}} \cdot v]\}$$

Method

IV. Implementation

- Training loss

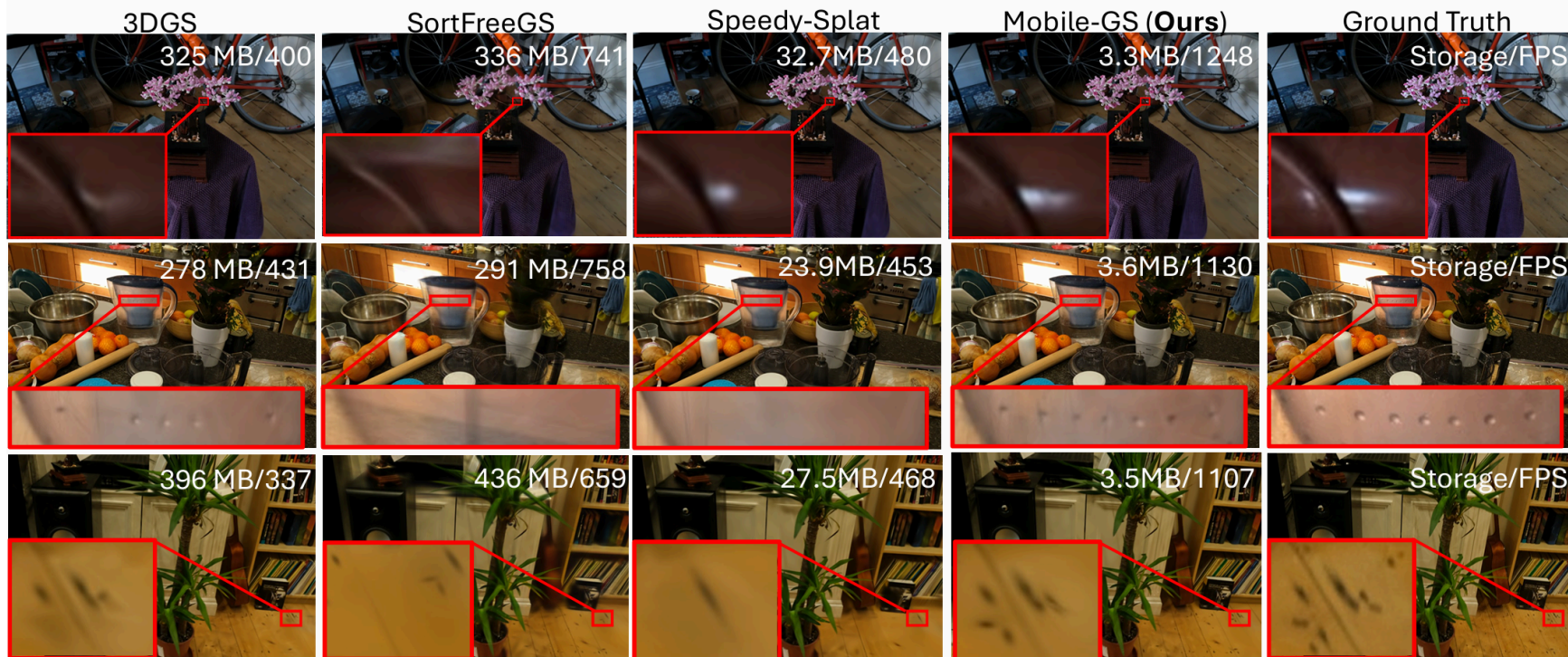
$$\mathcal{L}_{\text{rgb}} = \lambda \mathcal{L}_1(\mathbf{C}, \mathbf{C}_{\text{gt}}) + (1 - \lambda) \mathcal{L}_{\text{DSSIM}}(\mathbf{C}, \mathbf{C}_{\text{gt}})$$

$$\mathcal{L} = \mathcal{L}_{\text{rgb}} + \lambda_{\text{distill}} \mathcal{L}_{\text{distill}} + \lambda_{\text{depth}} \mathcal{L}_{\text{depth}}$$

- Training details
 - RTX 3090 GPU
 - 60k iterations
 - 35k-th iteration에서 NVQ 시작
 - Mobile devices : Snapdragon 8 Gen 3 GPU

Experiments

Dataset Method & Metrics	Mip-NeRF360					Tanks&Temples					Deep Blending				
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Storage $\downarrow$	FPS $\uparrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Storage $\downarrow$	FPS $\uparrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Storage $\downarrow$	FPS $\uparrow$
3DGS Kerbl et al. 2023	27.21	0.815	0.214	839.9 MB	174	23.14	0.841	0.183	371.5 MB	236	29.41	0.903	0.243	697.3 MB	214
LightGaussian Fan et al. 2024	27.08	0.801	0.244	60.4 MB	227	22.61	0.803	0.264	29.9 MB	392	28.74	0.856	0.325	48.2 MB	271
AdR-Gaussian Wang et al. 2024a	26.95	0.792	0.259	358.2 MB	254	22.74	0.809	0.251	214.6 MB	372	28.92	0.863	0.305	251.4 MB	284
SortFreeGS Hou et al. 2025	27.02	0.775	0.267	851.4 MB	731	22.81	0.817	0.254	471.5 MB	848	28.69	0.852	0.326	724.2 MB	793
Speedy-Splat Hanson et al. 2025	26.92	0.782	0.296	79.4 MB	401	23.08	0.821	0.241	62.4 MB	527	29.11	0.864	0.309	71.2 MB	463
C3DGS Lee et al. 2024	27.03	0.797	0.247	30.6 MB	184	23.32	0.831	0.202	21.8 MB	174	29.73	0.900	0.258	24.7 MB	189
LocoGS-S Shin et al. 2025	27.02	0.805	0.241	8.5 MB	292	23.23	0.837	0.204	6.8 MB	325	29.76	0.903	0.251	7.8 MB	322
Mobile-GS (Ours)	27.12	0.807	0.235	4.6 MB	1125	23.09	0.831	0.208	2.5 MB	1179	29.93	0.906	0.243	4.6 MB	1132



Experiments

Table 2: **Evaluation on the mobile device with Snapdragon 8 Gen 3 GPU.** 3DGS*, Mini-Splatting*, and SortFreeGS* mean the quantized version through Huffman encoding.

Method	PSNR $\uparrow$	FPS* $\uparrow$	Storage $\downarrow$	Training $\downarrow$
3DGS*	27.01	8	61.8 MB	0.5 h
Mini-Splatting*	27.02	12	36.9 MB	0.4 h
Speedy-Splat	26.92	19	79.5 MB	0.4 h
HAC	26.98	12	11.8 MB	0.7 h
LocoGS-S	27.02	17	8.5 MB	0.8 h
C3DGS	27.03	14	30.6 MB	0.6 h
GES	26.98	18	29.4 MB	0.7 h
SortFreeGS*	26.74	24	64.3 MB	1.3 h
Mobile-GS (Ours)	27.12	127	4.6 MB	1.5 h

Table 3: **Ablation Study of the proposed components.** We report results on the Mip-NeRF 360 dataset. The inference speed FPS is evaluated on the Desktop RTX 3090 GPU.

Method	PSNR $\uparrow$	FPS $\uparrow$	Storage $\downarrow$
Mobile-GS	27.12	1125	4.6 MB
w/o order-independent	27.26	684	4.5 MB
w/o view-dependent	26.68	1227	4.4 MB
w/o neural quantization	27.33	841	121 MB
w/ 0th-order SH distill.	27.04	1219	3.6 MB
w/ 2nd-order SH distill.	27.13	917	7.3 MB
w/ 3rd-order SH	27.15	841	9.6 MB
w/o depth in Eq.3	27.03	1167	4.5 MB
w/o scale in Eq.3	27.08	1171	4.5 MB

Experiments

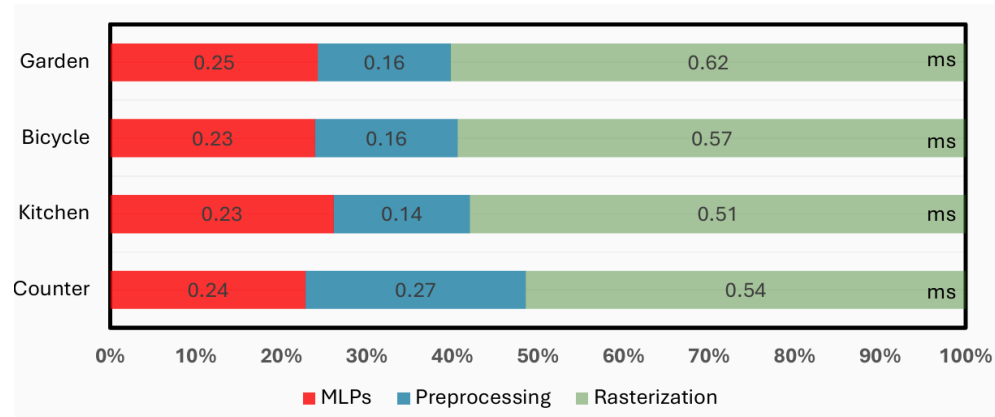


Figure 7: **Evaluation of the proposed neural view-dependent enhancement strategy.** “w/o” and “w/” mean the removal and the integration of the proposed neural view-dependent enhancement strategy.

Experiments

Table 4: **Ablation study of pruning strategies.** We leverage our Mobile-GS without pruning as the baseline and leverage pruning only on opacity, scale, and both attributes.

Method	Baseline	Opacity	Scale	Opacity & Scale
Num. $\times 10^6 \downarrow$	0.56	0.43	0.45	0.47
PSNR $\uparrow$	27.22	26.84	26.87	27.12
FPS* $\uparrow$	109	135	132	127

Table 6: **Adaptivity of the proposed contribution-based pruning.** Our proposed contribution-based pruning can be applied in MaksGaussian (Liu et al., 2025) and Mini-Splatting for further Gaussian pruning.

Method	MaskGaussian	+ prune.	Mini-Splatting	+ prune.
PSNR $\uparrow$	27.24	27.16	27.41	27.38
Num. $\times 10^6 \downarrow$	1.21	0.84	0.58	0.47

Table 5: **Hyperparameter analysis about the pruning threshold.** We employ our Mobile-GS without our contribution-based pruning as the baseline and adjust the pruning threshold to find a suitable trade-off.

Threshold	Baseline	0.1	0.2	0.4	0.6
Num. $\times 10^6 \downarrow$	0.56	0.55	0.47	0.34	0.18
PSNR $\uparrow$	27.22	27.15	27.12	26.47	25.85
FPS* $\uparrow$	109	111	127	141	164

Table 7: **Analysis of the codebook size.** We analyze different codebook sizes on the Mip-NeRF 360 dataset to find a more balanced trade-off. The smaller codebook size means fewer storage costs.

Codebook size	2^6	2^8	2^{10}	2^{12}
PSNR $\uparrow$	25.52	26.83	27.12	27.15
Storage $\downarrow$	3.84 MB	4.2 MB	4.6 MB	7.9 MB

Conclusion

- Mobile-GS는 3D Gaussian Splatting을 모바일에서 실시간으로 렌더링할 수 있도록 하는 방법론
- Order-independent rendering, distillation과 quantization, contribution-based pruning을 활용하여, 모바일에서 실시간 렌더링이
 - 경량화에 할 수 있는 모든걸 다함
- 빠르다, 퀄리티도 꽤 좋다, 압축 열심히 하고 품질 유지한것도 좋다.
- 다만 bounded scene이다.
- incremental하게 확장하는걸 좋아하는 사람은 어떻게 활용할 수 있을까
- 꼭 MLP를 활용해야하나? geometric한 방법으로 더 빠르게 할 수는 없나?